



Procedural Language / Structured Query Language

PL/SQL is a combination of SQL along with procedural features of programming language.

PL/SQL is a block structure meaning that

PL/SQL programs are divided and written in logical blocks of code

BLOCK → 3 sections

Declarations

(optional)

Starts with

DECLARE &

defines variables, cursors, subprograms & other elements used in the programs

Execute Commands (mandatory)

→ enclosed b/w Keywords BEGIN and END

→ consists of executable PL/SQL statements of the program.

→ It should have at least one executable line of code, w/c may be just a NULL command to indicate nothing should be executed.

Exception Handling

(optional)

→ Starts with keyword EXCEPTION

→ contains exceptions that handle the errors in the program.

* Every PL/SQL Statement ends with a semicolon
(;)

BASIC STRUCTURE OF A PL/SQL BLOCK

Structure

DECLARE

<declaration section>

BEGIN

<executable commands>

EXCEPTION

<Exception handling>

END;

or default keyword
message varchar2(20) DEFAULT
'Have a good day';
or
hello

Example

// DECLARE

message

varchar2(20) (:=) 'Hello World';

Assignment operator

BEGIN

similar to dbms_output.put_line(message);
variable

END;

PL/SQL USER DEFINED SUBTYPES

Subtype → subset of another datatype
(w/c is called its
base type)

PL/SQL defines various subtypes in
package STANDARD (having subtypes)

name of
subtype

→ Basotype

不

A

5

subtypes

name

mess

base types

name

message

BEGIN

salutation := 'Reader';

Salutation := 'Hello' || Salutation ||

greetings: = welcome a
elems - output . put - line ('Hello' || Salutation ||
greetings); concatenation

concatenation
operator

END;

SUBPROGRAM

A subprogram is a program that performs a particular task.

These subprograms are combined to form larger programs. This is called a modular design.

A subprogram can be invoked by another subprogram or program.

PL/SQL Subprograms

Functions

- these subprograms return a single value.
- mainly used to compute or return a value.

Procedures

- these subprograms do not return a value directly.
- mainly used to perform action.

A Subprogram can be created

At scheme level

- It is a standalone program.
- It is created with CREATE PROCEDURE or CREATE FUNCTION statement.
- It is stored in a database & can be deleted with DROP PROCEDURE or DROP FUNCTION.

Inside a package

- It is called a package subprogram.
- It is stored in database & can be deleted only when the package is deleted with DROP PACKAGE statement.

Inside a PL/SQL block

Syntax

CREATING A PROCEDURE

procedure-name

allows modifying an existing procedure

```
CREATE [OR REPLACE] PROCEDURE procedure-name  
[ (parameter name [IN | OUT | IN OUT] type [, ...])]   
{ IS | AS }   
↓   
optional
```

BEGIN

< procedure body >

END procedure-name;

Example

```
CREATE PROCEDURE greetings  
AS
```

BEGIN

dbms_output.put_line('Hello world');

END;

Executing a procedure

```
BEGIN
```

greetings;

END;

~~* EXECUTE greet;~~

Deleting a procedure

```
DROP PROCEDURE greetings;
```

BEGIN

~~DROP PROCEDURE greetings;~~

~~END;~~

PARAMETER MODES IN PL/SQL PROGRAMS

IN

→ lets you pass a value to the

subprogram

→ read-only parameter

→ Parameters are passed by reference

OUT

→ returns a value to the calling program.

→ the actual parameter must be variable & it is passed by value.

IN OUT

→ passes an initial value to a subprogram & returns an updated value to the caller.

→ Actual parameter is passed by value

EXAMPLE OF IN & OUT MODE
Program finds minimum of two values, procedure takes 2 numbers using IN mode & returns their minimum using OUT parameters.

DECLARE

a number;

b number;

c number;

PROCEDURE

findMin (x IN number, y IN number, z OUT number) IS

BEGIN

IF x < y THEN

z := x;

ELSE

z := y;

END IF;

END;

BEGIN

a := 23;

b := 45;

findMin (a, b, c);

obms - output, put - line ('Min of (23, 45): ' || c);

END;

Output:

Minimum of (23, 45) : 23

EXAMPLE 2

DECLARE

a. number;

PROCEDURE squareNum (x IN OUT number) IS

BEGIN

x := x * x;

END;

BEGIN

a := 23;

squareNum(a);

obms - output, put - line ('Square of (23): ' || a);

END;

Output:

Square of (23) : 529

FUNCTIONS

A function is same as a procedure except that it returns a value.

SYNTAX

```
CREATE [OR REPLACE] FUNCTION function-name  
[ ( parameter-name [IN|OUT | IN OUT] type [, ...] ) ]  
RETURN return-datatype  
{ IS | AS }  
BEGIN  
    <function-body>  
END [function-name];
```

EXAMPLE : - creating & calling a function.

The function returns the total number of employees in employee table.

```
select * from emp;           // displaying table
```

```
CREATE FUNCTION totalEmp // creating a function  
RETURN number IS
```

```
total number(2) := 0;
```

```
BEGIN
```

```
SELECT count(*) into total FROM emp;
```

```
RETURN total;
```

```
END;
```


DECLARE

c number(2);

BEGIN

c := totalEmp();

dbms_output.put_line('Total no. of employees: ' || c);

END;

Output

Total no. of employees : 14

// calling a function

CURSORS

Oracle creates a memory area, known as context area, for processing an SQL statement which contains all information needed for processing the statement. eg. number of rows processed etc.

A cursor is a pointer to this context area. PL/SQL controls the context area through a cursor.

A cursor holds the rows (one or more) returned by a SQL statement.

The set of rows the cursor holds is referred to as the active set.

We can name a cursor so that it could be referred to in a program to fetch and process the rows returned by the SQL statement, one at a time.

TYPES OF CURSORS

IMPLICIT CURSORS

EXPLICIT CURSORS

IMPLICIT CURSORS

→ Automatically created by Oracle whenever an SQL statement is executed.

→ We cannot control the implicit cursors and its information in it.

→ Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement.

For INSERT → the cursor holds the data that needs to be inserted.

UPDATE & DELETE → cursor identifies the rows that would be affected.

→ Most recent implicit cursor as SQL cursor.

The following attributes are :-

1) %FOUND - returns TRUE if an INSERT, UPDATE or DELETE statement is affected.
(It identifies if the data is present or not).

OR a SELECT INTO statement returned one or more rows.

Otherwise, it returns FALSE.

2) %NOTFOUND - logical opposite of %FOUND

→ It returns TRUE if an INSERT, UPDATE or DELETE statement affected no rows or a SELECT INTO statement returned no rows.

Otherwise, it returns FALSE.

3) %ISOPEN - Always returns FALSE for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.

4) %ROWCOUNT - Returns the number of rows affected by an INSERT, UPDATE or DELETE statement, or returned by a SELECT INTO statement.

→ Any SQL cursor attribute will be accessed as sql%attribute-name

EXAMPLE

= select * from emp;

The program will update the table and increase salary of each customer by 500 and use the SQL%ROWCOUNT attribute to determine the number of rows affected:

DECLARE

total_row number(2);

BEGIN

UPDATE emp SET salary = salary + 500;

IF sql%notfound THEN:

dbms_output.put_line('no ^{employees} customers selected');

← ELSEIF sql%found THEN

total_rows := sql%rowcount;

dbms_output.put_line(total_rows || '^{employees} customers selected');

END IF;

END;

EXPLICIT CURSORS

- Explicit cursors are user defined cursors for gaining more control over the context area.
- An explicit cursor should be defined in the declaration section of the PL/SQL block.
- It is created on a SELECT statement which returns more than one row.

SYNTAX for creating explicit cursor is

CURSOR cursor-name IS select-statement ;

Working with an explicit cursor involves 4 steps :-

- 1) Declaring the cursor for initializing in the memory.
- 2) Opening the cursor for allocating memory.
- 3) Fetching the cursor for retrieving data.
- 4) Closing the cursor to release allocated memory.

DECLARING THE CURSOR

Declaring the cursor defines the cursor with a name and the associated SELECT statement.

Eg. CURSOR (cemp) ^{→ cursor name} IS SELECT empid, ename, address
FROM emp;

OPENING THE CURSOR

OPEN c-emp;

FETCHING THE CURSOR

FETCH c-emp INTO c-id, c-name, c-addr;

CLOSING THE CURSOR

CLOSE c-emp;

EXAMPLE

DECLARE
// c-id emp.empid%type;

variable declared in cursor
table name
means that the datatype of these variables will be as that from the table
column name in table

DECLARE
c-id emp.empno%type;
c-name emp.ename%type;
c-job emp.job%type;

table
column
global

CURSOR c-emp IS

SELECT empno, ename, job FROM emp;

BEGIN

OPEN c-emp; // Step 1: Declaring cursor for initializing in the memory

LOOP:

FETCH c-emp INTO c-id, c-name, c-job; // Step 2: opening cursor
dems-output.put-line (c-id || ' ' || c-name ||
" || c-job); // Step 3: fetching cursor

EXIT WHEN c-emp%notfound;

END LOOP;

CLOSE c-emp; // Step 4: closing the cursor

END;

TRIGGERS

- Triggers are stored programs, which are automatically executed or fired when some event occurs.
- Triggers can be defined on table, view, schema or database with which the event is associated.

SYNTAX FOR CREATING TRIGGERS

✓ CREATE [OR REPLACE] TRIGGER trigger-name → creates or replaces an existing trigger with the trigger name

{ BEFORE | AFTER | INSTEAD OF } → Specifies when the trigger will be executed. [Instead of]

{ INSERT [OR] | UPDATE [OR] | DELETE } → DML operations is used to create trigger on view.

{ OF col name } → columnname that would be updated

ON table name → name of table associated with trigger

[REFERENCING OLD AS o NEW AS n]

[FOR EACH ROW] → Allows to refer new & old values for various DML statements

WHEN (condition) → specifies a row level trigger i.e. trigger would be executed for each row being affected. otherwise the trigger will execute just once when the SQL statement is executed, which is called level trigger.

DECLARE Declaration - statements

BEGIN Executable - statements

EXCEPTION Exception - handling - statements

END;

Provides conditions for rows, this is valid for only row level triggers.

EXAMPLE

select * from emp;

1000

2000

3000

4000

1000

Program to create a row level trigger for employee table that would fire for INSERT or UPDATE or DELETE operations performed on Employee table. This trigger will display the salary difference b/w the old values & new values.

CREATE TRIGGER display-salary-changes

BEFORE DELETE OR INSERT OR UPDATE ON emp-02,
FOR EACH ROW

WHEN (NEW.empno > 0)

DECLARE

sal-diff number;

BEGIN

sal-diff := :NEW.sal - :OLD.sal;

dbms_output.put_line ('Old salary: ' || :OLD.sal);

dbms_output.put_line ('New salary: ' || :NEW.sal);

dbms_output.put_line ('Salary difference: ' || sal-diff);

END;

→ Insert a new record in the table employee

insert into emp values (1234, 'Aman', 'Manager', 1111,
'12-04-81', 1200, ' ', 20);

output

old salary: NULL

new salary: 1200

Salary difference:

* → old salary is blank because it is not available as it is a new record so the result is coming null.

→ Updating an Existing record.

Update emp set sal = sal + 500 where

empno = 1234;

→ Trigger will be fired.

→ output

old salary = 1200

new salary: 1700

Salary difference: 500

TO DELETE A TRIGGER

DROP TRIGGER <Triggername>

AFTER → Indicates that trigger is fired after executing the triggering statement.

BEFORE → Indicates that trigger is fired before executing the triggering statement.

SYNTAX

→ Adding a new column in an existing table

Alter table <tablename> add <columnname> <datatype>;

→ Drop a column in an existing table.

Alter table <tablename> drop column <columnname>;

CONSTRAINTS

- 1) Primary key
- 2) Not null
- 3) check
- 4) default
- 5) foreign key

→ Add / Drop a primary key

Alter table <tablename> add constraint- ^{name} <myconstraint>
primary key (<columnname>);

Alter table <tablename> drop primary key;

→ Add / Drop Not Null constraint-

Alter table <tablename> modify <columnname>
<datatype> not null;

Alter table <tablename> modify <columnname>
<datatype> null;

→ Add / Drop check constraint-

Alter table <tablename> add constraint-
<columnname / constraintname> check (condition);

INDEXES

When a SELECT statement is fired to search for a particular record, the database engine first locates the table on the HD Hard Disk.

System information

finds the start location of a table's record on the current storage media.

1) The database engine performs a sequential search to locate records to match user defined criteria as specified in the SELECT.

⇒ Indexing a table is an access strategy.
A way to sort & search records in the table.

→ Indexes are essential to improve the speed with which record can be located & retrieved from the table.

An index is an ordered list of contents of a column, (or group of columns) in a table.

Indexing involves forming a 2D matrix completely independent of the table on which the index is being created.

2D matrix $\rightarrow$ single column $\rightarrow$ w/c will hold the sorted data.

extracted from table column on w/c the index is created

Another column is called the address field $\rightarrow$ identifies the location of the record in the Oracle database.

working when data is inserted in the table

$\rightarrow$ Oracle engine automatically inserts the data value in the index.

$\downarrow$ for every data value Oracle inserts a unique RowID $\rightarrow$ indicates exactly where the record is stored in the table.

INDEX RowID: return

Select rowid, empno from emp-02;

VIEWS

- After a table is created & populated with data, it may become necessary to prevent all users from accessing all columns of a table, for data security reasons.
- This would mean creating several tables having the appropriate number of columns and assigning specific users to each table as required.
- This will answer data security requirements very well but will give rise to great deal of redundant data being resident in tables, in the database.